



UNIVERSITÀ DI PISA

DEPARTMENT OF INFORMATION ENGINEERING (DII)

Master Degree in Cybersecurity

Malware Analysis - Dependability

Group #5:

Francesco Copelli
(Mat. 675686)

Contents

0.1	Background: the Malbus Family	4
0.2	Tools	5
1	Malbus_v1.0.3.apk	6
1.1	VirusTotal	6
1.1.1	Detection	6
1.1.2	Details	8
1.1.3	Relations	9
1.1.4	Behaviour	10
1.2	Static Analysis	10
1.2.1	MainActivity.java	10
1.2.2	BaseMainActivity.java	11
1.2.3	EnvConstant.java	11
1.2.4	BusApiWeb.java	11
1.2.5	AndroidManifest.xml	12
1.2.6	AppInfoActivity.java	12
1.3	Summary	12
1.4	MobSF	13
1.4.1	Certificate Analysis	13
1.4.2	High Severity Findings	13
1.4.3	Medium Severity Findings	14
1.4.4	Low Severity Findings	14
1.4.5	Overall Assessment	14
1.5	Genymotion	14
1.6	Conclusion	14
2	Malbus_v3.3.7.apk	17
2.1	VirusTotal	17
2.1.1	Detection	17
2.1.2	Details	17
2.1.3	Relations	19
2.1.4	Behavior	20
2.2	Static Analysis	20
2.2.1	MainActivity.java	20
2.2.2	BaseMainActivity.java	20
2.2.3	EnvConstant.java	21
2.2.4	BusApiWeb.java	21
2.2.5	AndroidManifest.xml	21
2.2.6	AppInfoActivity.java	22
2.3	Summary	22
2.4	MobSF	23
2.4.1	Certificate Analysis	23
2.4.2	High Severity Findings	23
2.4.3	Medium Severity Findings	23
2.4.4	Low Severity Findings	24

2.4.5	Overall Assessment	24
2.5	Genymotion	24
2.6	Conclusion	24
3	Malbus_v3.6.5.apk	27
3.1	VirusTotal	27
3.1.1	Detection	27
3.1.2	Details	27
3.1.3	Relations	29
3.1.4	Behavior	30
3.2	Static Analysis	31
3.2.1	MainActivity.java	31
3.2.2	BaseMainActivity.java	31
3.2.3	EnvConstant.java	31
3.2.4	BusApiWeb.java	32
3.2.5	AndroidManifest.xml	32
3.2.6	AppInfoActivity.java	33
3.3	Summary	33
3.4	MobSF	33
3.4.1	Certificate Analysis	34
3.4.2	High Severity Findings	34
3.4.3	Medium Severity Findings	34
3.4.4	Low Severity Findings	34
3.4.5	Overall Assessment	35
3.5	Genymotion	35
3.6	Conclusion	35
4	Comparative Analysis	37
4.1	VirusTotal Detection	37
4.2	Permissions and Attack Surface	37
4.3	Behavioural Tags and Evasion Capabilities	38
4.4	Static Analysis: Common Architecture	38
4.5	C2 Infrastructure Status	39
4.6	MobSF Static Analysis Comparison	39
4.6.1	High-Severity Findings	40
4.6.2	Medium-Severity Findings	40
4.7	YARA Rule Matches	41
4.8	Overall Family Assessment	41

Introduction

The aim of this project is to study the behaviour of three different versions of the same application for Android devices through antimalware, static, and dynamic analysis. The main goal was to identify the malicious payload inside the APK files of the provided samples.

The project is composed of three different files:

1. 3252fbcee2d1aff76a9f18b858231adb741d4dc07e803f640dcbbab96db240f9.apk, (from now on it will be "Malbus_v1.0.3.apk"), see Chapter 1
2. 19162b063503105fdc1899f8f653b42d1ff4fcfdcf261f04467fad5f563c0270.apk, (from now on it will be "Malbus_v3.3.7.apk"), see Chapter 2
3. bed3e665d2b5fd53aab19b8a62035a5d9b169817adca8dfb158e3baf71140ceb.apk, (from now on it will be "Malbus_v3.6.5.apk"), see Chapter 3

0.1 Background: the Malbus Family

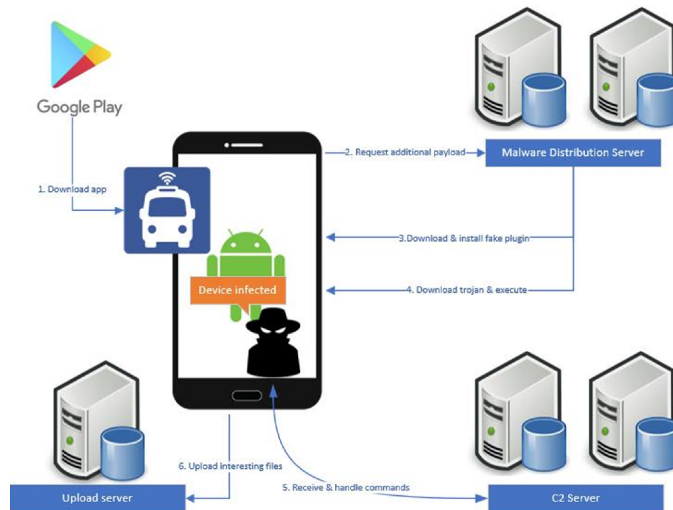


Figure 1: Malbus infection chain as documented by McAfee Mobile Research

The three samples analysed in this project belong to a malware family publicly documented by McAfee researchers under the name **Malbus**. The family was first described in a McAfee Mobile Research report detailing a series of trojanised Android applications masquerading as legitimate bus-information services targeting South Korean users. The malicious apps were distributed on the Google Play Store and, once installed, silently contacted remote servers to download and execute additional payloads on the victim device.

The McAfee-documented infection chain, illustrated in Fig. 1, unfolds in six stages. The user first downloads the apparently legitimate bus app from Google Play (step 1). The app then contacts a Malware Distribution Server to request an additional payload (step 2), which is delivered and

installed on the device as a fake plug-in (step 3). A second download delivers and executes the core trojan (step 4). From this point on, the infected device communicates bidirectionally with a Command-and-Control (C2) server to receive and handle commands (step 5), while a separate Upload Server is used to exfiltrate files of interest collected from the device (step 6).

The three APK samples provided for this project correspond to different versions of the same repackaged application series (`cwbus`, `gjbus`, `jjbus`), each targeting a different South Korean city. They represent successive releases of the Malbus dropper, allowing for a comparative analysis of how the malicious infrastructure and evasion techniques evolved across versions. The findings of the individual analyses are consolidated in Chapter 4.

0.2 Tools

During the analysis, three main analysis tools have been used to identify malicious behaviour of the samples.

- VirusTotal [<https://www.virustotal.com/>] (anti-malware analysis)
 - It is a web tool that allows to submit samples and analyse them with several antivirus or anti-malware programmes
 - This tool was used to gain an initial insight into the already existing knowledge about the specific malicious sample.
- MobSF [<https://www.github.com/MobSF/Mobile-Security-Framework-MobSF/>] (static and dynamic analysis)
 - It automatically highlights interesting features of the application (e.g. Android permissions, API calls, remote URLs). It also extracts the Java code from the APK file. In this way we can gain a strong insight of the potential malicious behavior of the application and then manually analyze it by examining the code.
 - Moreover, it performs a dynamic analysis, by executing the application inside a virtual environment and by monitoring it.
- Java Decompiler [<https://www.java-decompiler.github.io/>]
 - This tool allows users to displays Java source codes of ".class" files graphically. It can browse the reconstructed source code with the Java Decompiler Graphical User Interface (GUI) for instant access to methods and fields.
- Genymotion [<https://www.genymotion.com/>] (dynamic analysis)
 - Creates a virtual environment for MobSF in order to execute the malware sample safely.

Chapter 1

Malbus_v1.0.3.apk



What is it? The Malbus_v1.0.3.apk is a malicious Android application masquerading as a plugin for a transportation application series developed by a South Korean developer. The series provides a range of information for each region of South Korea, such as bus stop locations, bus arrival times and so on.

1.1 VirusTotal

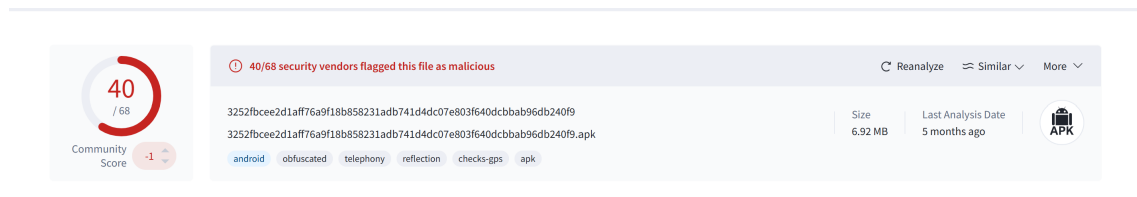


Figure 1.1: Virustotal summary analysis of file Malbus_v1.0.3.apk

1.1.1 Detection

The antimalware analysis performed with VirusTotal, shown in fig. 1.2, highlights that:

- 40 out of 75 security vendors flagged the file as “**malicious**”;
- 28 out of 75 security vendors flagged the file as a “**secure file**”;
- 7 out of 75 security vendors get an “**Unable to process file type**” error.

Most anti-malware programs correctly identify the sample as a generic Android trojan, but some of them are even able to recognise its malware family (downloader). A few vendors classify the sample as a generic Android malware, without specifying the family nor the type (trojan). It is also noteworthy that some well-known vendors, including Microsoft Defender, McAfee, and Malwarebytes, did not identify the file as malicious.

VirusTotal assigns the sample a reputation score of -1 and a suggested threat label of `trojan.andr/apkdownloader`. The popular threat classification aggregated across all detecting engines identifies “trojan” as the dominant category (18 engines), followed by “downloader” (5 engines). Notably, the engine Tencent identifies the sample under the specific name `a.privacy.MalBuslmh`, a designation that directly references the APK package name `lmh.android.cwbus`, confirming that this specific sample — rather than a generic variant — is known to the threat intelligence community.

Crowdsourced YARA rules			
Matches rule IcedID from ruleset IcedID at https://github.com/kevoreilly/CAPEv2 by kevoreilly, threatlive ↳ IcedID Payload View Ruleset			
Popular threat label trojan.piom/andr	Threat categories trojan downloader		Family labels piom andr apkdownloader
Security vendors' analysis		Do you want to automate checks?	
AhnLab-V3	Trojan.Android.FakeKakao.851419	AliCloud	Trojan[downloader].Android.Agent.NN
ALYac	Trojan.Linux.Generic.102427	Arcabit	Trojan.Linux.Generic.D2013646 [many]
Avast	ELF:ApkDownloader-D [Drp]	Avast-Mobile	Android:Evo-gen [Trj]
AVG	ELF:ApkDownloader-D [Drp]	Avira (no cloud)	LINUX/AVL.Agent.liuwj
BitDefender	Trojan.Linux.GenericKD.33633862	BitDefenderFalx	Android.Trojan.Agent.LACO
ClamAV	Unix.Malware.Agent-7050989-0	CTX	Apk.trojan.generic
Cynet	Malicious (score: 99)	DrWeb	Android.DownLoader.4452
Emsisoft	Trojan.Linux.GenericKD.33633862 (B)	eScan	Trojan.Linux.GenericKD.33633862
ESET-NOD32	Multiple Detections	Fortinet	ELF/Agent.NYitr
GData	Trojan.Linux.Generic.102427	Google	Detected
Ikarus	Trojan-Downloader.AndroidOS.Agent	K7GW	Trojan (005514201)
Kaspersky	HEUR:Trojan.AndroidOS.Piom.aafh	Kingsoft	Android.Troj.TrojanDownlo_JVYBELN
Lionic	Trojan.AndroidOS.Piom.Clc	NANO-Antivirus	Trojan.ELFArm64.Dwn.fqtdff
QuickHeal	Cld.android.piom.1646380100	Rising	Spyware.Lazer/Android.IB.10817 (TFE:20:...
Sangfor Engine Zero	Suspicious.Linux.Save.a	Skyhigh (SWG)	GenericRXTG-DUI0420199A95CD
Sophos	Andr/Dloadr-EEL	Symantec	Trojan.Gen.MBT
Symantec Mobile Insight	Other.Android.Reputation.1	Tencent	A.privacy.MalBuslmh
Trellix (ENS)	ArtemisI056886BFC9F4	Trustlook	Android.Malware.Trojan
Varist	E64/DCAgent.FTJK-1	VIPRE	Trojan.Linux.GenericKD.33633862
WithSecure	Malware.ANDROID/Agent.FRFV.Gen	Xcitium	Malware@#kicichew3bfff
Zillya	Downloader.Agent.Android.16123	ZoneAlarm by Check Point	Andr/Dropr-IG
Acronis (Static ML)	Undetected	Alibaba	Undetected
Antiy-AVL	Undetected	Baidu	Undetected
Bkav Pro	Undetected	CMC	Undetected
CrowdStrike Falcon	Undetected	Elastic	Undetected
Gridinsoft (no cloud)	Undetected	Huorong	Undetected
Jiangmin	Undetected	K7AntiVirus	Undetected
Malwarebytes	Undetected	MaxSecure	Undetected
McAfee Scanner	Undetected	Microsoft	Undetected
Panda	Undetected	SentinelOne (Static ML)	Undetected
SUPERAntiSpyware	Undetected	TACHYON	Undetected
TrendMicro	Undetected	TrendMicro-HouseCall	Undetected
VBA32	Undetected	VirIT	Undetected
ViRobot	Undetected	Webroot	Undetected
Yandex	Undetected	Zoner	Undetected
Arctic Wolf	Unable to process file type	DeepInstinct	Unable to process file type
Palo Alto Networks	Unable to process file type	SecureAge	Unable to process file type
TEHTRIS	Unable to process file type	Trapmine	Unable to process file type

Figure 1.2: VirusTotal Detection tab for Malbus_v1.0.3.apk

1.1.2 Details

Permissions	
	android.permission.ACCESS_FINE_LOCATION
	android.permission.READ_EXTERNAL_STORAGE
	com.android.browser.permission.WRITE_HISTORY_BOOKMARKS
	android.permission.ACCESS_COARSE_LOCATION
	android.permission.WRITE_EXTERNAL_STORAGE
	lmh.android.cwbus.permission.C2D_MESSAGE
	android.permission.VIBRATE
	com.google.android.providers.gsf.permission.READ_GSERVICES
	android.permission.ACCESS_LOCATION_EXTRA_COMMANDS
	com.google.android.c2dm.permission.RECEIVE
	android.permission.ACCESS_NETWORK_STATE
	android.permission.WAKE_LOCK
	android.permission.CHANGE_WIFI_STATE
	android.permission.ACCESS_WIFI_STATE
	android.permission.INTERNET

Figure 1.3: Android Permission of Malbus_v1.0.3.apk

Here is a detailed analysis of the permissions (Fig. 1.3) which are required by the APK, declared in the Manifest file (Fig. 1.4):

1. **android.permission.ACCESS_FINE_LOCATION**→ Allows the precise location of the device (GPS, mobile networks, Wi-Fi).
2. **android.permission.ACCESS_COARSE_LOCATION**→ Provides approximate location (based on cellular towers and Wi-Fi).
3. **android.permission.ACCESS_LOCATION_EXTRA_COMMANDS**→ Allows additional commands to be sent to the location provider (rarely used in legitimate apps).
4. **android.permission.READ_EXTERNAL_STORAGE**→ Allows reading files and data saved on external memory (photos, documents, downloads).
5. **android.permission.WRITE_EXTERNAL_STORAGE**→ Allows you to create or edit files on external storage.
6. **com.android.browser.permission.WRITE_HISTORY_BOOKMARKS**→ Allows editing of history and bookmarks in the stock Android browser.
7. **com.google.android.providers.gsf.permission.READ_GSERVICES**→ Reads Google Services Framework settings and data (e.g. device ID, GCM configurations).
8. **lmh.android.cwbus.permission.C2D_MESSAGE**→ Enable reception of cloud-to-device messages (C2DM/GCM/FCM).
9. **com.google.android.c2dm.permission.RECEIVE**→ Allows push notifications via Google Cloud Messaging (GCM/FCM).
10. **android.permission.INTERNET**→ Enables any network connection (HTTP, sockets, DNS).
11. **android.permission.ACCESS_NETWORK_STATE**→ Reads the status of network connections (Wi-Fi, mobile data).
12. **android.permission.ACCESS_WIFI_STATE**→ Gets information about Wi-Fi networks (SSID, BSSID, status).

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android" android:versionCode="4" android:versionName="1.0.3" package="lmh.android.cwbus"
platformBuildVersionCode="24" platformBuildVersionName="7">
  <uses-sdk android:minSdkVersion="14" android:targetSdkVersion="25"/>
  <permission android:name="lmh.android.cwbus.permission.C2D_MESSAGE" android:protectionLevel="signature"/>
  <uses-permission android:name="android.permission.INTERNET"/>
  <uses-permission android:name="android.permission.ACCESS_LOCATION_EXTRA_COMMANDS"/>
  <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
  <uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
  <uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
  <uses-permission android:name="android.permission.VIBRATE"/>
  <uses-permission android:name="android.permission.WAKE_LOCK"/>
  <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
  <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
  <uses-permission android:name="lmh.android.cwbus.permission.C2D_MESSAGE"/>
  <uses-permission android:name="com.google.android.providers.gsf.permission.READ_GSERVICES"/>
  <uses-permission android:name="com.android.browser.permission.WRITE_HISTORY_BOOKMARKS"/>
  <uses-permission android:name="com.google.android.c2dm.permission.RECEIVE"/>
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>

```

Figure 1.4: Android Manifest Malbus_v1.0.3.apk

13. **android.permission.CHANGE_WIFI_STATE**→ Changes Wi-Fi status (enable/disable, connects to networks).
14. **android.permission.WAKE_LOCK**→ Prevents the device from entering sleep mode.
15. **android.permission.VIBRATE**→ Controls the vibration engine.

In addition to the permission analysis, VirusTotal assigns several behavioural and structural tags to the file: **obfuscated**, **telephony**, **reflection**, **checks-gps**, **android**, and **apk**. The tags **obfuscated** and **reflection** are consistent with the use of Java Reflection observed during static analysis, while **checks-gps** directly confirms that the application programmatically queries the GPS subsystem at runtime. The **telephony** tag further indicates interaction with device communication APIs, which is not expected for a bus-information application and may be relevant to tracking or device fingerprinting.

1.1.3 Relations

Contacted URLs (4) ⓘ			
Scanned	Detections	Status	URL
2018-03-16	0 / 67	200	http://busappmanager.elasticbeanstalk.com/v1/category/list.json
2022-09-09	0 / 88	200	http://busappmanager.elasticbeanstalk.com/v1/env/get.json
2018-03-16	0 / 67	200	http://busappmanager.elasticbeanstalk.com/v1/issue/my.json
2023-07-17	0 / 90	200	http://gw.k.adlibr.com/ad/config/init

Contacted Domains (3) ⓘ			
Domain	Detections	Created	Registrar
busappmanager.elasticbeanstalk.com	0 / 94	2011-01-04	MarkMonitor Inc.
github.com	0 / 94	2007-10-09	MarkMonitor Inc.
gw.k.adlibr.com	0 / 94	2012-04-04	Gabia, Inc.

Contacted IP addresses (7) ⓘ			
IP	Detections	Autonomous System	Country
172.217.214.95	0 / 94	15169	US
172.67.151.52	0 / 94	13335	-
173.194.193.95	0 / 94	15169	US
173.194.195.95	0 / 94	15169	US
209.85.145.99	0 / 94	15169	US
64.233.179.101	0 / 94	15169	US
74.125.201.99	0 / 94	15169	US

Figure 1.5: VirusTotal Relations Tab Malbus_v1.0.3.apk

In addition, VirusTotal shows the contacted domains, URLs, and IP addresses associated with the APK, as shown in Fig. 1.5. The first three URLs are hosted on AWS Elastic Beanstalk (**busappmanager.elasticbeanstalk.com**) and appear to retrieve the app category, runtime environment configuration, and remote download URL from a central database. This mechanism is consistent with *dropper-like* behaviour, as it allows the app to modify its functionality without

requiring a new release. The fourth domain (`gwk.adlibr.com/ad/config/init`) serves the advertising configuration and is still reachable; it responds only to POST requests, otherwise returning `{ "result": 4003, "message": "key_media_key" }`. The bus-related domains (`bus.changwon.go.kr`, `openAPIs.changwon.go.kr`) are legitimate Korean public APIs, and no flagged domains were detected.

VirusTotal also records several alternative names under which the sample has been submitted across its 377 total submissions: `malbus.apk`, `supplybus.apk`, `lmh.android.cwbus-1.0.3.apk`, and others. The variety of filenames under which the same hash appears suggests that the sample has been distributed or repackaged under different labels, which is consistent with trojanised application behaviour.

Additionally, a notable anomaly is present in the `AndroidManifest`: the `GcmReceiver` component is registered under the category `lmh.android.dgbus`, which differs from the main application package `lmh.android.cwbus`. This discrepancy in package naming for a core push-messaging receiver may indicate the presence of a secondary plugin or dropper component operating under a separate namespace.

1.1.4 Behaviour

VirusTotal's behaviour tab highlights several actions consistent with a suspicious application profile. The APK accesses file-related resources, builds JSON objects from file names and absolute paths, reads data into buffer streams, and opens files from absolute paths. It also shows the use of reflection/control to start another application from the current one, which may indicate dynamic or indirect execution of external components. In addition, the app interacts with advertising and analytics libraries, scheduling recording tasks and reading files from the assets directory. Overall, the observed behaviour suggests a complex application that performs file-system interaction and may support remote or dynamic actions rather than a simple bus-information client.

Furthermore, a crowdsourced YARA rule match was recorded for this sample: the rule `IcedID`, from the ruleset `IcedID` (author: `kevoreilly/threathive`, source: CAPEv2 on GitHub), matched the binary. While `IcedID` is historically associated with a Windows banking trojan, the matching rule in this context is a structural pattern rule targeting dropper and loader architectures. The match does not imply that the sample belongs to the `IcedID` family, but it does indicate that its internal structure shares characteristics with known advanced payload-delivery components, further reinforcing the dropper hypothesis.

1.2 Static Analysis

The Java source code was decompiled using JD-GUI / Bytecodeviewer and inspected manually to identify classes and methods of interest. Static analysis reveals that the application combines legitimate bus-related functionality with advertising SDKs, remote configuration mechanisms, native code, and update-like behavior. These elements suggest that the APK is not limited to its declared purpose, but may also support conditional execution and payload delivery at runtime.

1.2.1 MainActivity.java

- Inherits from `BaseMainActivity`.
- Calls `getTabView(BaseTabFragmentActivity.MainTabs.BUSTRANS).setVisibility(8)` to hide a UI tab.
- Provides ad network keys through `getAdvertiseKey(...)`.
- Retrieves configuration via `getEnvironmentProperties(...)`.

The analysis of `MainActivity` suggests that the visible interface is partially controlled at runtime. In particular, the hiding of a UI tab and the retrieval of external configuration parameters indicate that the application may adapt its behavior dynamically depending on remotely supplied values. The use of multiple advertising providers such as AdMob, Facebook, and AdLib further confirms the presence of monetization-oriented logic.

1.2.2 BaseMainActivity.java

- Acts as the central initialization hub.
- Loads the native library with `System.loadLibrary("Audio3.0")`.
- Declares and invokes the native methods `startUpdate()` and `updateApplication()`.
- Requests the runtime permissions:
 - `READ_EXTERNAL_STORAGE`
 - `WRITE_EXTERNAL_STORAGE`
 - `REQUEST_INSTALL_PACKAGES`
- Loads remote app environment data, including a URL, a database version, and a remote download URL.

This class is the most relevant part of the static analysis. The presence of a native library, together with methods related to update handling and the request for `REQUEST_INSTALL_PACKAGES`, is consistent with dropper-like behavior. In addition, the remote environment loading mechanism allows the application to modify its behavior without requiring a new release.

1.2.3 EnvConstant.java

```
public static final String ADAMKEY = "3034Z1OT1385cf1db09";
public static final String ADLIBKEY = "590ea88a0cf2843c93807ed9";
public static final String ADMOBUNITID = "ca-app-pub-0388605861855854/7499990747";
public static final String FACEBOOK = "627046304133735228304842";
public static final String GA_TRACKER_ID = "UA-56668425-6";
```

The constants defined in this class confirm the integration of several advertising and analytics platforms. The presence of hardcoded keys is relevant from a security perspective, because it reduces transparency and can facilitate reverse engineering or unauthorized reuse of the embedded services. Although no endpoint URLs are directly visible in this file, the constants are consistent with the remote configuration logic observed in the main activity.

1.2.4 BusApiWeb.java

- Makes calls to:
 - `http://bus.changwon.go.kr/busInfo/busInfoDetail.jsp`
 - `http://bus.changwon.go.kr/busInfo/mapLayer.jsp`
- Uses Volley for networking.
- Parses HTML responses instead of JSON.
- Relays information through listeners.

The main issue in this component is the exclusive use of HTTP. Since the communication is not protected by TLS, bus stop identifiers and route data travel in plaintext and can be intercepted or modified in transit. This creates a clear man-in-the-middle risk and weakens the integrity of the information delivered by the application.

1.2.5 AndroidManifest.xml

- Declares the following permissions:
 - `INTERNET`
 - `ACCESS_FINE_LOCATION`
 - `ACCESS_COARSE_LOCATION`
 - `READ_EXTERNAL_STORAGE`
 - `WRITE_EXTERNAL_STORAGE`
 - `REQUEST_INSTALL_PACKAGES`
- Declares multiple ad and analytics activities:
 - AdLib
 - AdMob
 - Facebook
 - AdFit (Kakao)
- Includes GCM and Firebase messaging support.
- Declares the custom service `lmh.android.cwbus.GCMIntentService`.

The manifest confirms that the application can receive background messages and interact with external services. In particular, the presence of `REQUEST_INSTALL_PACKAGES` is significant because, if granted, it may allow silent installation of additional APKs. Combined with GCM/Firebase messaging, this suggests the possibility of remote triggering of behavior.

1.2.6 AppInfoActivity.java

- Displays the link `http://bus.changwon.go.kr/main`.
- Serves to legitimize the app.
- Contains no interaction logic.

This activity appears to be mainly cosmetic, since it exposes a public bus-related URL while providing no additional functional logic. Its role seems to be presenting the application as a legitimate transportation service, even though the rest of the static analysis reveals a much more complex and suspicious architecture.

1.3 Summary

Suspicious Pattern	Explanation
Multiple ad SDKs	Monetization through aggressive advertising and possible ad fraud.
Native libraries (JNI)	Executes code not visible to standard Java analysis tools.
<code>REQUEST_INSTALL_PACKAGES</code>	Suggests dropper-like behavior and possible APK installation without market interaction.

Unencrypted HTTP endpoints	Data can be intercepted or modified through MITM attacks.
Dynamic configuration via <code>getEnvironmentProperties(...)</code>	The app can change behavior based on remote values.
Fragment and tab visibility manipulation	Suggests conditional or delayed functionality and may hide behavior from users and analysts.
VT behavioural tags (obfuscated, telephony, checks-gps)	Confirm runtime evasion, device tracking via GPS, and telephony API interaction beyond the declared app purpose.
YARA match: IcedID rule	Structural similarity with known dropper/loader patterns detected via crowd-sourced YARA; reinforces the hypothesis of an advanced payload-delivery architecture.

1.4 MobSF

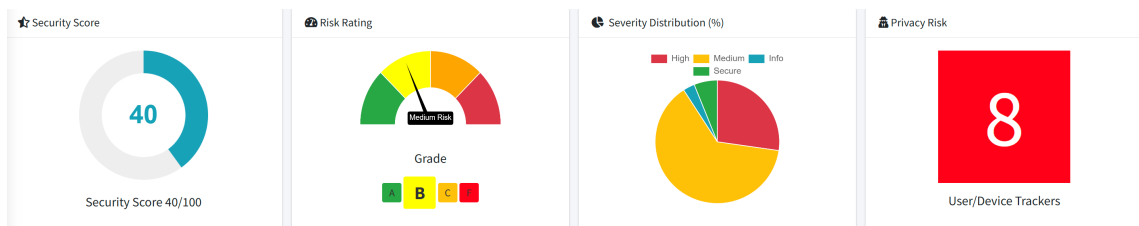


Figure 1.6: MobSF Scorecard Malbus_v1.0.3.apk

MobSF highlights several security issues in the analyzed APK, confirming that the application exposes a broad attack surface. In particular, the scorecard shows problems related to signature schemes, outdated Android compatibility, exported components, insecure storage, and the presence of multiple privacy trackers.

1.4.1 Certificate Analysis

During the analysis, MobSF identified one unique certificate. The binary is signed only with the v1 signature scheme, while v2, v3, and v4 are not present. The certificate uses `SHA1withRSA`, with the following subject and issuer: `C=ko, L=Jeonju, CN=Myoung-Hun Lee`. Since the app relies on a legacy signing scheme and a weak hash algorithm, it is exposed to known cryptographic and platform-level weaknesses.

1.4.2 High Severity Findings

MobSF reports nine high-severity issues. The first one is the vulnerability to the Janus attack, because the application is signed only with the v1 scheme. The second is the use of SHA-1, which is vulnerable to hash collision attacks. The third is the low minimum SDK version (`minSdk=14`), which allows installation on older Android versions with unresolved vulnerabilities. MobSF also detects Android Task Hijacking and StrandHogg weaknesses in `lmh.android.cwbus.activity.AlarmActivity` and in `com.google.android.gms.appinvite.PreviewActivity`. In addition, the analysis flags

debug configuration, weak encryption in the AdLib library, a world-readable file or shared preference, and the presence of more than eight privacy trackers.

1.4.3 Medium Severity Findings

The medium-severity findings are mainly related to insecure configuration and component exposure. The flag `android:allowBackup=true` makes the application data backup accessible through ADB when USB debugging is enabled. Several activities and receivers are exported or not properly protected, including `AlarmActivity`, `CampaignTrackingReceiver`, `FirebaseMessagingService`, `TagManagerPreviewActivity`, and Firebase/GCM-related components. MobSF also warns about the external permission associated with `INSTALL_PACKAGES`, which is particularly relevant because it may support silent installation behavior.

The scan also identifies insecure storage and code-quality issues. The application can read and write external storage, which exposes data to other apps on the device. MobSF further detects hardcoded secrets in multiple files, insecure WebView usage, temporary file creation, an insecure random number generator, possible SQL injection through raw queries, and the use of MD5, which is considered weak due to collision resistance problems.

1.4.4 Low Severity Findings

MobSF reports one informational issue: the application logs information, including potentially sensitive data. This finding is classified as LOW severity in this variant; as will be shown in Chapter 3, the same behaviour is elevated to HIGH severity in `Malbus_v3.6.5.apk` due to a broader set of sensitive fields being exposed in that version. It also reports two secure findings: the use of SSL communication and the presence of root detection capabilities. Finally, the scorecard flags one hotspot related to a critical permission, reinforcing the idea that the app is highly sensitive from a security perspective.

1.4.5 Overall Assessment

Overall, MobSF confirms that the APK is not a simple transportation application. The combination of weak signing, exported components, insecure storage, dynamic installation capabilities, and ad/tracking infrastructure suggests an application with a high security risk and behavior consistent with a trojanized or heavily repurposed app.

1.5 Genymotion

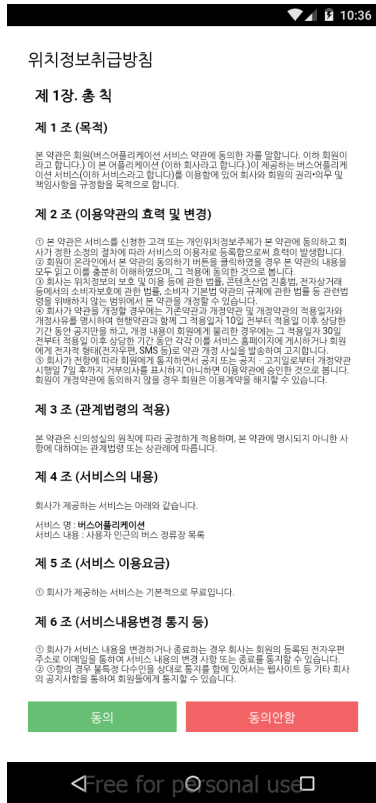
The dynamic analysis performed in Genymotion provided practical confirmation of several findings obtained during static inspection. The application interacts with remote endpoints, relies on advertising frameworks, and appears capable of adapting part of its behavior through external configuration data rather than through local code changes.

This behavior is relevant because it indicates that the app is not limited to a simple transportation interface. Instead, it combines user-facing functionality with background services, ad-related activity, and network-driven logic, which is consistent with a repackaged or trojanized application structure.

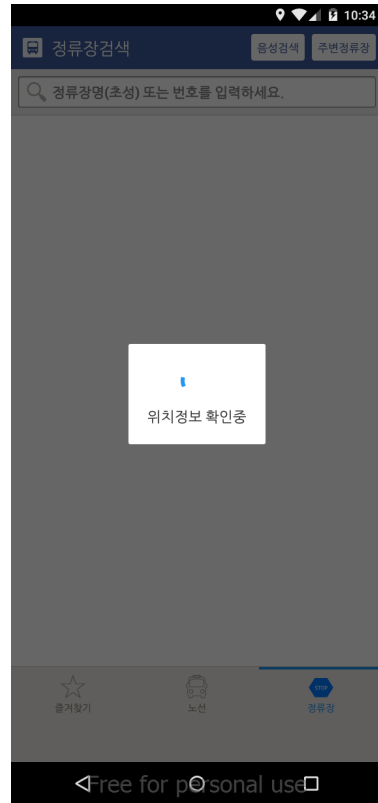
The emulator analysis therefore strengthens the overall assessment of the sample as suspicious. In combination with the insecure HTTP traffic, exported components, hardcoded secrets, and multiple privacy trackers identified by other tools, the Genymotion observations support the conclusion that the APK presents a non-negligible security risk.

1.6 Conclusion

The combined results of VirusTotal, static analysis, MobSF, and Genymotion consistently indicate that `Malbus_v1.0.3.apk` is suspicious and potentially malicious. All four analytical tools converge on the same core patterns: unencrypted network traffic, aggressive integration of advertising libraries,



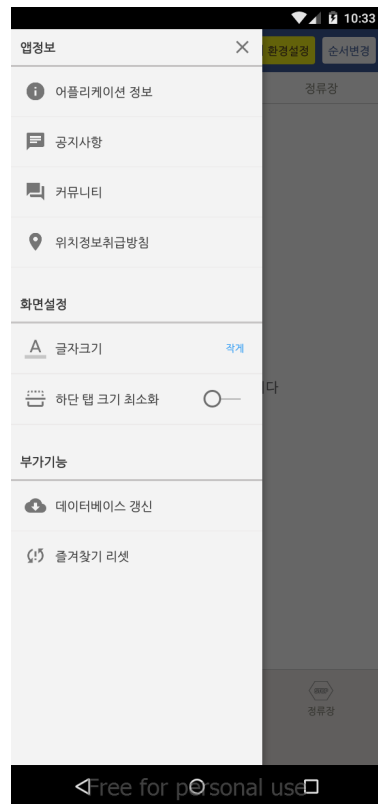
(a) Privacy policy / consent screen shown at app startup
Malbus_v1.0.3.apk



(b) Runtime location verification screen in Genymotion
Malbus_v1.0.3.apk



(c) Main bus search screen showing the DB error during
dynamic analysis Malbus_v1.0.3.apk



(d) Side menu with maintenance options such as
database refresh and favorites reset Malbus_v1.0.3.apk

Figure 1.7: Genymotion dynamic analysis screenshots of Malbus_v1.0.3.apk

remote configuration via `getEnvironmentProperties()`, exported components, and weaknesses related to storage and cryptography.

The static inspection reveals that the application extends well beyond a legitimate bus-information interface: the presence of a native library loaded at startup, native methods `startUpdate()` and `updateApplication()` consistent with dropper logic, and a cosmetic `AppInfoActivity` designed to disguise the application's true nature are all characteristic of repackaged or trojanised software. MobSF reinforces this picture by reporting nine high-severity issues, including Janus vulnerability exposure, SHA-1 signing, StrandHogg weaknesses, hardcoded secrets, and multiple privacy trackers. The dynamic analysis in Genymotion confirms that the app contacts remote services and relies on background components whose behaviour is driven at runtime rather than being fully encoded in the static bytecode.

At this stage, the evasion layer is limited to code obfuscation and Java Reflection — the techniques recorded by the VirusTotal tags `obfuscated` and `reflection`. No sandbox or debugger detection is present in this variant, which suggests that `Malbus_v1.0.3.apk` represents the baseline form of the family before active anti-analysis hardening was introduced in later releases. The APK should be treated as unsafe: the combination of insecure communication, suspicious permissions, and evasive implementation choices reveals a clear mismatch between the application's declared purpose and its actual security posture.

Chapter 2

Malbus_v3.3.7.apk



What is it? The *Malbus_v3.3.7.apk* sample is a malicious Android application masquerading as a plug-in for a transportation app series developed by a South Korean developer. The legitimate series provides bus-related information, such as bus stop locations and arrival times, which makes the repackaged sample appear trustworthy at first glance. The package name `lmh.android.gjbus` and the internal name `gjbus` are consistent with a localised variant of the same Malbus family, targeting the Gwangju metropolitan area.

2.1 VirusTotal

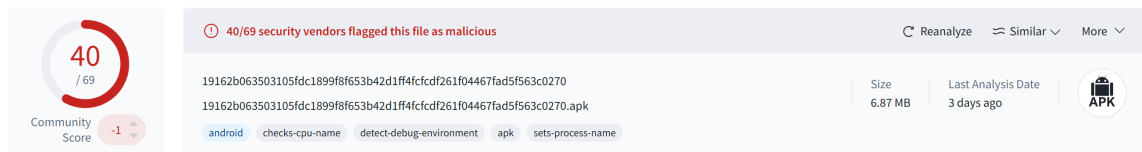


Figure 2.1: VirusTotal Summary for Malbus_v3.3.7.apk

2.1.1 Detection

The antimalware analysis performed with VirusTotal shows that 40/75 security vendors flagged the file as malicious, while 28/75 vendors classified it as a secure file, 1/75 reported a failure error, and 6/75 vendors reported an *Unable to process file type* error. Most engines identify the sample as a generic Android trojan, and some of them further classify it as a downloader-family specimen. A few vendors instead label it only as generic Android malware, without specifying a precise family. It is also noteworthy that some well-known vendors, including Microsoft Defender, McAfee, and Malwarebytes, did not identify the file as malicious.

VirusTotal assigns the sample a reputation score of -2 and a suggested threat label of `trojan.andr/apkdownload`. The popular threat classification aggregated across all detecting engines identifies “trojan” as the dominant category (17 engines), followed by “downloader” (4 engines) and “dropper” (1 engine). Notably, the engine Tencent identifies the sample under the specific name `a.privacy.MalBuslmh`, a designation that directly references the APK package namespace `lmh`, confirming that this specific sample is known to the threat intelligence community.

2.1.2 Details

Here is a detailed analysis of the permissions which are required by the APK, declared in the Manifest file (Fig. 2.4):

1. **android.permission.ACCESS_FINE_LOCATION** → Allows the precise location of the device (GPS, mobile networks, Wi-Fi).

`checks-cpu-name`, and `sets-process-name` are explicit anti-analysis and VM evasion indicators, consistent with the same pattern observed in the third sample. The `telephony` tag further indicates interaction with device communication APIs, which is not expected for a bus-information application.

2.1.3 Relations

Contacted URLs (5) ⓘ			
Scanned	Detections	Status	URL
2025-05-23	0 / 97	-	http://busappmanager.elasticbeanstalk.com/v1/category/list.json
2022-09-09	0 / 88	200	http://busappmanager.elasticbeanstalk.com/v1/env/get.json
2018-07-05	0 / 68	404	http://smartapps.cafe24.com/gjbus_android/environment2.json
2018-03-16	0 / 67	200	http://busappmanager.elasticbeanstalk.com/v1/issue/my.json
2025-05-23	0 / 97	200	http://gwk.adlibr.com/ad/config/init

Contacted Domains (8) ⓘ			
Domain	Detections	Created	Registrar
adlibr.com	0 / 94	2012-04-04	Gabia, Inc.
api.snapcraft.io	0 / 94	2016-05-16	MarkMonitor Inc.
busappmanager.elasticbeanstalk.com	0 / 94	2011-01-04	MarkMonitor Inc.
cafe24.com	0 / 94	1999-05-21	Gabia, Inc.
elasticbeanstalk.com	0 / 94	2011-01-04	MarkMonitor Inc.
github.com	1 / 94	2007-10-09	MarkMonitor Inc.
gwk.adlibr.com	0 / 94	2012-04-04	Gabia, Inc.
smartapps.cafe24.com	0 / 94	1999-05-21	Gabia, Inc.

Contacted IP addresses (4) ⓘ			
IP	Detections	Autonomous System	Country
185.125.188.54	0 / 94	41231	GB
185.125.188.55	0 / 94	41231	GB
185.125.188.58	0 / 94	41231	GB
185.125.188.59	0 / 94	41231	GB

Figure 2.3: VirusTotal Relations for Malbus_v3.3.7.apk

VirusTotal, as shown in Fig. 2.3, reports multiple contacted URLs, domains, and IP addresses associated with the APK. The contacted URLs are all served over plain HTTP:

- <http://busappmanager.elasticbeanstalk.com/v1/category/list.json>
- <http://busappmanager.elasticbeanstalk.com/v1/env/get.json>
- <http://busappmanager.elasticbeanstalk.com/v1/issue/my.json>
- http://smartapps.cafe24.com/gjbus_android/environment2.json
- <http://gwk.adlibr.com/ad/config/init>

The first three URLs are hosted on AWS Elastic Beanstalk and appear to retrieve the app category list, runtime environment configuration, and remote issue data from a centralised backend. This mechanism is consistent with dropper-like behaviour, as it allows the app to modify its functionality without requiring a new APK release. The fourth URL (smartapps.cafe24.com) retrieves a configuration file specific to this `gjbus` variant, using a path (`/gjbus_android/environment2.json`) that distinguishes it from the other regional Malbus samples. The domain gwk.adlibr.com serves the advertising configuration; it responds only to POST requests, otherwise returning `{"result":4003,"message":"key_media_key"}`.

VirusTotal also records several alternative names under which the sample has been submitted across its 354 total submissions: `bus.apk`, `malbus.apk`, `lmh.android.gjbus.apk`, `00081.apk`, and others. The variety of filenames under which the same hash appears suggests that the sample has been distributed or repackaged under different labels, which is consistent with trojanised application behaviour.

2.1.4 Behavior

The behaviour tab indicates that the APK accesses file-related resources, builds JSON objects from filenames and absolute paths, reads data through buffer streams, and opens files from absolute paths. It also shows the use of reflection or control-based execution to start another application from the current one, which may indicate indirect execution of external components. In addition, the sample interacts with advertising and analytics libraries, schedules recording tasks, and reads files from the assets directory.

Notably, this variant carries explicit **anti-analysis** and **VM evasion** indicators consistent with the third Malbus sample. VirusTotal tags the APK with **detect-debug-environment**, indicating that the application actively probes for debugger and sandbox contexts at runtime. The tag **checks-cpu-name** reveals hardware fingerprinting consistent with virtual machine detection, and **sets-process-name** indicates process name manipulation to obfuscate the running process from monitoring tools.

Overall, the observed behaviour is more complex than that of a simple bus-information client and is compatible with a repackaged application that supports file-system interaction, dynamically driven actions, and active evasion of analysis environments. Additionally, as observed in the first Malbus sample, a crowdsourced YARA rule match was recorded for this sample: the IcedID structural rule from the CAPEv2 repository matched the binary. As discussed in Chapter 1, this does not imply family membership but confirms structural similarity with known dropper and loader architectures.

2.2 Static Analysis

The Java source code was decompiled using JD-GUI / Bytecodeviewer and inspected manually to identify classes and methods of interest. Static analysis shows that the application combines legitimate bus-related functionality with advertising SDKs, remote configuration mechanisms, native code, and update-like behaviour. This indicates that the APK is not limited to its declared purpose, but may also support conditional execution and payload delivery at runtime.

2.2.1 MainActivity.java

- Inherits from `BaseMainActivity`.
- Calls `getTabView(BaseTabFragmentActivity.MainTabs.BUSTRANS).setVisibility(8)` to hide a UI tab.
- Provides ad-network keys through `getAdvertiseKey(...)`.
- Retrieves configuration through `getEnvironmentProperties(...)`.

This combination suggests that the visible interface is at least partially controlled at runtime. In particular, the hiding of a UI tab and the retrieval of external configuration parameters imply that the application may adapt its behaviour according to remotely supplied values. The use of multiple advertising providers, including AdMob, Facebook, and AdLib, further confirms the presence of monetization-oriented logic.

2.2.2 BaseMainActivity.java

- Acts as the central initialization hub.

- Loads the native library with `System.loadLibrary("Audio3.0")`.
- Declares and invokes the native methods `startUpdate()` and `updateApplication()`.
- Requests the runtime permissions:
 - `READ_EXTERNAL_STORAGE`
 - `WRITE_EXTERNAL_STORAGE`
 - `REQUEST_INSTALL_PACKAGES`
- Loads remote app environment data, including a URL, a database version, and a remote download URL.

This class is the most relevant part of the static analysis. The presence of a native library, together with methods related to update handling and the request for `REQUEST_INSTALL_PACKAGES`, is consistent with dropper-like behaviour. In addition, the remote environment loading mechanism allows the application to modify its behaviour without requiring a new release.

2.2.3 EnvConstant.java

The `EnvConstant` class defines several hardcoded advertising and analytics keys. MobSF confirmed the extraction of hardcoded secrets including `adpostkey`, `adlibkey`, and hash keys embedded directly in the source code.

```
public static final String ADAMKEY = "3034Z1OT1385cf1db09";
public static final String ADLIBKEY = "590ea88a0cf2843c93807ed9";
public static final String ADMOB_UNITID = "ca-app-pub-0388605861855854/7499990747";
public static final String FACEBOOK = "627046304133735685335228304842";
public static final String GA_TRACKER_ID = "UA-56668425-6";
```

These constants confirm the integration of multiple ad SDKs and tracking frameworks. Although no endpoint URLs are directly visible in this file, the constants are consistent with the remote-configuration logic observed in the main activity.

2.2.4 BusApiWeb.java

`BusApiWeb` performs network requests to bus-related endpoints over plain HTTP. It uses Volley for networking, parses HTML responses instead of JSON, and relays information through listeners.

- Makes calls to `http://bus.changwon.go.kr/busInfo/busInfoDetail.jsp` and `http://bus.changwon.go.kr/`.
- Uses Volley for networking.
- Parses HTML responses instead of JSON.
- Relays information through listeners.

The main issue in this component is the exclusive use of HTTP. Since the communication is not protected by TLS, bus-stop identifiers and route data travel in plaintext and can be intercepted or modified in transit, creating a clear man-in-the-middle risk.

2.2.5 AndroidManifest.xml

- Declares the following permissions:
 - `INTERNET`, `ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`
 - `READ_EXTERNAL_STORAGE`, `WRITE_EXTERNAL_STORAGE`
 - `REQUEST_INSTALL_PACKAGES`
- Declares multiple ad and analytics activities:

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android" android:versionCode="62" android:versionName="3.3.7" package="lmh.android.gjbus"
platformBuildVersionCode="24" platformBuildVersionName="7">
  <uses-sdk android:minSdkVersion="14" android:targetSdkVersion="25"/>
  <permission android:name="lmh.android.gjbus.permission.C2D_MESSAGE" android:protectionLevel="signature"/>
  <uses-permission android:name="android.permission.INTERNET"/>
  <uses-permission android:name="android.permission.ACCESS_LOCATION_EXTRA_COMMANDS"/>
  <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
  <uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
  <uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
  <uses-permission android:name="android.permission.VIBRATE"/>
  <uses-permission android:name="android.permission.WAKE_LOCK"/>
  <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
  <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
  <uses-permission android:name="com.google.android.c2dm.permission.RECEIVE"/>
  <uses-permission android:name="com.google.android.providers.gsf.permission.READ_GSERVICES"/>
  <uses-permission android:name="com.android.browser.permission.WRITE_HISTORY_BOOKMARKS"/>
  <uses-permission android:name="lmh.android.gjbus.permission.C2D_MESSAGE"/>

```

Figure 2.4: AndroidManifest of Malbus_v3.3.7.apk

– AdLib, AdMob, Facebook, AdFit (Kakao)

- Includes GCM and Firebase messaging support.
- Declares the custom service `lmh.android.gjbus.GCMIntentService`.

The manifest confirms that the application can receive background messages and interact with external services. In particular, the presence of `REQUEST_INSTALL_PACKAGES` is significant because, if granted, it may allow silent installation of additional APKs. Combined with GCM/Firebase messaging, this suggests the possibility of remote triggering of behaviour.

2.2.6 AppInfoActivity.java

- Displays a public bus-related URL for the Gwangju city service.
- Serves to legitimize the app.
- Contains no interaction logic.

This activity appears to be mainly cosmetic. Its role seems to be presenting the application as a legitimate transportation service, even though the rest of the static analysis reveals a much more complex and suspicious architecture.

2.3 Summary

Suspicious Pattern	Explanation
Multiple ad SDKs	Monetization through aggressive advertising and possible ad fraud.
Native libraries (JNI)	Executes code not visible to standard Java analysis tools.
<code>REQUEST_INSTALL_PACKAGES</code>	Suggests dropper-like behaviour and possible APK installation without market interaction.
Unencrypted HTTP endpoints	Data can be intercepted or modified through MITM attacks.
Dynamic configuration via <code>getEnvironmentProperties(...)</code>	The app can change behaviour based on remote values.
Fragment and tab visibility manipulation	Suggests conditional or delayed functionality and may hide behaviour from users and analysts.

VT behavioural tags (obfuscated, telephony, checks-gps)	Confirm runtime obfuscation, device tracking via GPS, and telephony API interaction beyond the declared app purpose.
Anti-analysis & VM evasion (detect-debug-environment, checks-cpu-name, sets-process-name)	The app actively probes for debuggers and sandboxes and manipulates its process name to evade monitoring tools; consistent with the third Malbus sample.
Alternative submission names (354 submissions)	bus.apk, malbus.apk, lmh.android.gjbus.apk confirm distribution under multiple filenames, consistent with trojanised repackaging.

2.4 MobSF

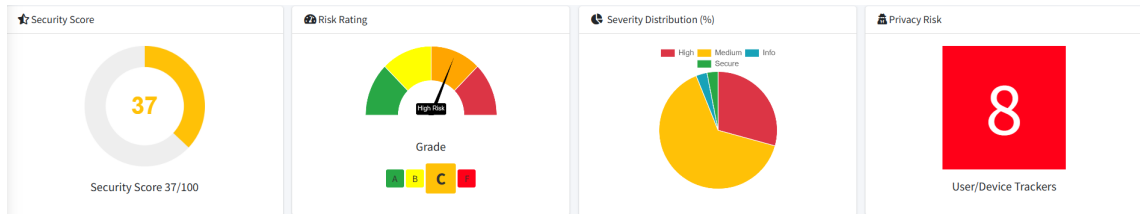


Figure 2.5: MobSF Scorecard Malbus_v3.3.7.apk

MobSF assigned a security score of 37/100 (Grade C, HIGH RISK) to this APK, confirming that the application exposes a broad attack surface. In particular, the scorecard shows problems related to signature schemes, outdated Android compatibility, exported components, insecure storage, and the presence of multiple privacy trackers.

2.4.1 Certificate Analysis

During the MobSF analysis, one unique certificate was found. The binary is signed with the v1 signature scheme only, while v2, v3, and v4 are absent. The certificate uses `SHA1withRSA`, with subject and issuer both identified as `C=ko, L=Gwangju, CN=Myoung-Hun Lee`. The validity interval spans from 2011-05-09 to 2061-04-26, and the reported hash algorithm is SHA-1. This signing configuration is weak from a security perspective because it relies on legacy mechanisms associated with known cryptographic and platform-level weaknesses, making it vulnerable to the Janus attack on Android 5.0–8.0.

2.4.2 High Severity Findings

MobSF reports nine high-severity issues. The first is vulnerability to the Janus attack, because the application is signed only with the v1 scheme. The second concerns SHA-1, which is vulnerable to hash-collision attacks. The third is the low minimum SDK version (`minSdk=14`), which allows installation on older Android versions with unresolved vulnerabilities. MobSF also detects Android Task Hijacking and StrandHogg weaknesses in `lmh.android.gjbus.activity.AlarmActivity` and `com.google.android.gms.appinvite.PreviewActivity`. In addition, the analysis flags debug configuration, weak encryption in the AdLib library, a world-readable file or shared preference, and the presence of more than eight privacy trackers.

2.4.3 Medium Severity Findings

The medium-severity findings are mainly related to insecure configuration and component exposure. The flag `android:allowBackup=true` makes application data accessible through ADB backups

when USB debugging is enabled. Several activities and receivers are exported or not properly protected, including `AlarmActivity`, `CampaignTrackingReceiver`, `FirebaseMessagingService`, `TagManagerPreviewActivity`, and Firebase/GCM-related components. MobSF also warns about the external permission associated with `INSTALL_PACKAGES`, which is especially relevant because it may support silent installation behaviour. The scan further identifies insecure storage and code-quality issues, including read/write access to external storage, hardcoded secrets in multiple files, insecure WebView usage, temporary file creation, an insecure random number generator, possible SQL injection through raw queries, and the use of MD5, which is considered weak because of collision-resistance problems.

2.4.4 Low Severity Findings

MobSF reports one informational issue: the application logs information, including potentially sensitive data. This finding is classified as LOW severity in this variant; as will be shown in Chapter 3, the same behaviour is elevated to HIGH severity in `Malbus_v3.6.5.apk` due to a broader set of sensitive fields being exposed in that version. It also reports two secure findings: the use of SSL communication and the presence of root detection capabilities. Finally, the scorecard flags one hotspot related to a critical permission, reinforcing the idea that the application is highly sensitive from a security perspective.

2.4.5 Overall Assessment

Overall, MobSF confirms that the APK is not a simple transportation application. The combination of weak signing, exported components, insecure storage, dynamic installation capabilities, and ad/tracking infrastructure suggests an application with a high security risk and behaviour consistent with a trojanized or heavily repurposed app.

2.5 Genymotion

The dynamic analysis performed in Genymotion provided practical confirmation of several findings obtained during static inspection. The application interacts with remote endpoints, relies on advertising frameworks, and appears capable of adapting part of its behaviour through external configuration data rather than through local code changes. This behaviour is relevant because it indicates that the app is not limited to a simple transportation interface. Instead, it combines user-facing functionality with background services, ad-related activity, and network-driven logic, which is consistent with a repackaged or trojanized application structure. The emulator analysis therefore strengthens the overall assessment of the sample as suspicious. In combination with insecure HTTP traffic, exported components, hardcoded secrets, and multiple privacy trackers identified by other tools, the Genymotion observations support the conclusion that the APK presents a non-negligible security risk.

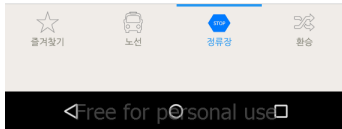
2.6 Conclusion

The combined results of VirusTotal, static analysis, MobSF, and Genymotion confirm that `Malbus_v3.3.7.apk` is malicious and shares the same core dropper architecture identified in `Malbus_v1.0.3.apk`. The structural patterns are consistent across both variants — native library loading, remote configuration via `getEnvironmentProperties()`, advertising SDK integration, insecure HTTP communication, and exported components — confirming family membership.

The defining characteristic of this variant, however, is the measurable evolution of its evasion profile. Unlike `Malbus_v1.0.3.apk`, which relies solely on code obfuscation and Java Reflection, `Malbus_v3.3.7.apk` introduces three additional behavioural tags absent from the previous variant: `detect-debug-environment`, `checks-cpu-name`, and `sets-process-name`. These indicate that the malware author deliberately hardened this release against dynamic analysis environments — probing for debuggers, fingerprinting the hardware to detect virtual machines, and manipulating



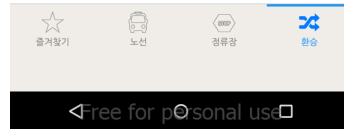
검색결과가 없습니다.



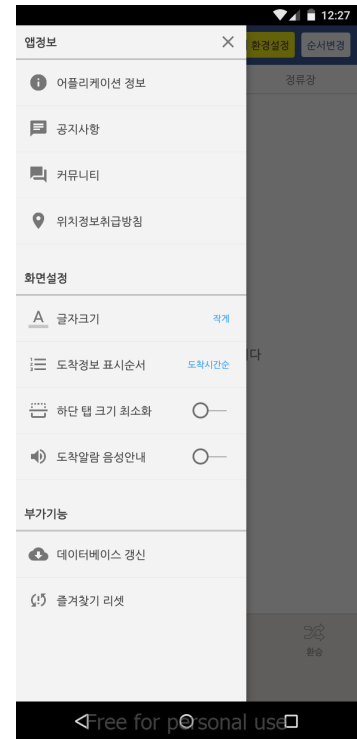
(a) Bus stop search screen showing no results during dynamic analysis Malbus_v3.3.7.apk



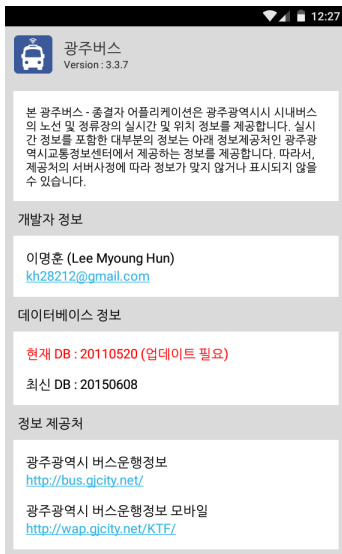
검색결과가 없습니다



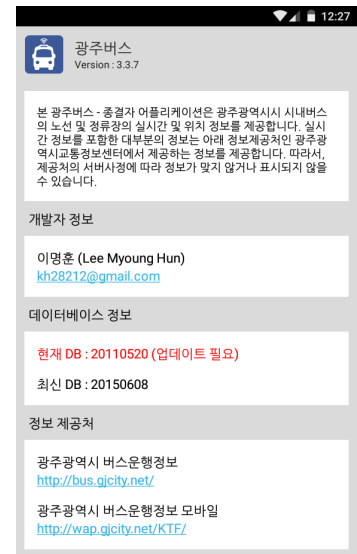
(b) Route search screen with departure and arrival fields showing no results Malbus_v3.3.7.apk



(c) App settings menu showing notification and display preferences Malbus_v3.3.7.apk



(d) App info screen showing outdated database state for Malbus_v3.3.7.apk



(e) Error pop-up indicating database download failure and prompting app restart Malbus_v3.3.7.apk

Figure 2.6: Genymotion dynamic analysis screenshots of Malbus_v3.3.7.apk

the process name to evade monitoring tools. This shift from passive obfuscation to active sandbox detection represents a qualitative step in the family’s anti-analysis sophistication.

MobSF confirms a high security risk with nine high-severity findings, including Janus vulnerability exposure, SHA-1 signing, StrandHogg weaknesses, and multiple privacy trackers — a profile identical in count and composition to `Malbus.v1.0.3.apk`. The dynamic analysis in Genymotion further confirms that runtime behaviour is partially driven by remote configuration, consistent with a dropper architecture designed to withhold its payload until the appropriate conditions are met. The APK should be treated as unsafe: its combination of insecure communication, suspicious permissions, and active anti-analysis evasion demonstrates a clear and deliberate mismatch between its declared purpose and its actual security posture.

Chapter 3

Malbus_v3.6.5.apk



What is it? The *Malbus_v3.6.5.apk* sample is a malicious Android application masquerading as a plug-in for a transportation app series developed by a South Korean developer. The legitimate series provides bus-related information, such as bus stop locations and arrival times, for the city of Jeonju, which makes the repackaged sample appear trustworthy at first glance. The package name `lmh.android.jjbus` and the internal name `jjbus` are consistent with a localised variant of the same Malbus family analysed in the previous chapters.

3.1 VirusTotal

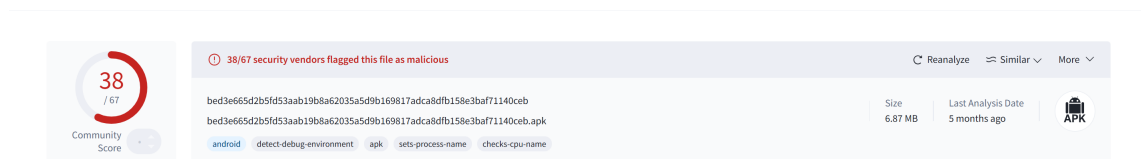


Figure 3.1: VirusTotal Summary for Malbus_v3.6.5.apk

3.1.1 Detection

The antimalware analysis performed with VirusTotal shows that 38/75 security vendors flagged the file as malicious, while 29/75 vendors classified it as a secure file, 1/75 reported a failure error, and 7/75 vendors reported an *Unable to process file type* error. Most engines identify the sample as a generic Android trojan, and some of them (e.g. CTX, Bitdefender, Arcabit) further classify it as a downloader-family specimen. A few vendors instead label it only as generic Android malware, without specifying a precise family. It is also noteworthy that some well-known vendors, including Microsoft Defender, McAfee, and Malwarebytes, did not identify the file as malicious.

VirusTotal assigns the sample a reputation score of 0 and a suggested threat label of `trojan.andr/apkdownloader`. The popular threat classification aggregated across all detecting engines identifies “trojan” as the dominant category (17 engines), followed by “downloader” (5 engines). Notably, the engine Tencent identifies the sample under the specific name `a.privacy.MalBuslmh`, a designation that directly references the APK package namespace `lmh`, confirming that this specific sample is known to the threat intelligence community.

3.1.2 Details

Here is a detailed analysis of the permissions which are required by the APK, declared in the Manifest file (Fig. 3.4):

Permissions

-  `android.permission.ACCESS_FINE_LOCATION`
-  `android.permission.READ_EXTERNAL_STORAGE`
-  `com.android.browser.permission.WRITE_HISTORY_BOOKMARKS`
-  `android.permission.ACCESS_COARSE_LOCATION`
-  `android.permission.WRITE_EXTERNAL_STORAGE`
-  `lmh.android.jjbus.permission.C2D_MESSAGE`
-  `android.permission.VIBRATE`
-  `com.google.android.providers.gsf.permission.READ_GSERVICES`
-  `android.permission.ACCESS_LOCATION_EXTRA_COMMANDS`
-  `com.google.android.c2dm.permission.RECEIVE`
-  `android.permission.ACCESS_WIFI_STATE`
-  `android.permission.WAKE_LOCK`
-  `android.permission.CHANGE_WIFI_STATE`
-  `android.permission.ACCESS_NETWORK_STATE`
-  `android.permission.INTERNET`

1. **`android.permission.ACCESS_FINE_LOCATION`** → Allows the precise location of the device (GPS, mobile networks, Wi-Fi).
2. **`android.permission.ACCESS_COARSE_LOCATION`** → Provides approximate location (based on cellular towers and Wi-Fi).
3. **`android.permission.ACCESS_LOCATION_EXTRA_COMMANDS`** → Allows additional commands to be sent to the location provider (rarely used in legitimate apps).
4. **`android.permission.READ_EXTERNAL_STORAGE`** → Allows reading files and data saved on external memory (photos, documents, downloads).
5. **`android.permission.WRITE_EXTERNAL_STORAGE`** → Allows you to create or edit files on external storage.
6. **`com.android.browser.permission.WRITE_HISTORY_BOOKMARKS`** → Allows editing of history and bookmarks in the stock Android browser.
7. **`com.google.android.providers.gsf.permission.READ_GSERVICES`** → Reads Google Services Framework settings and data (e.g. device ID, GCM configurations).
8. **`android.permission.INTERNET`** → Enables any network connection (HTTP, sockets, DNS).
9. **`android.permission.ACCESS_NETWORK_STATE`** → Reads the status of network connections (Wi-Fi, mobile data).
10. **`android.permission.ACCESS_WIFI_STATE`** → Gets information about Wi-Fi networks (SSID, BSSID, status).

Figure 3.2: VirusTotal Detection for Malbus_v3.6.5.apk

11. **android.permission.CHANGE_WIFI_STATE** → Changes Wi-Fi status (enable/disable, connects to networks).
12. **android.permission.WAKE_LOCK** → Prevents the device from entering sleep mode.
13. **android.permission.VIBRATE** → Controls the vibration engine.
14. **lmh.android.jjbus.permission.C2D_MESSAGE** → Enables reception of cloud-to-device messages (C2DM/GCM/FCM).
15. **com.google.android.c2dm.permission.RECEIVE** → Allows push notifications via Google Cloud Messaging (GCM/FCM).

In addition to the permission analysis, VirusTotal assigns several behavioural and structural tags to the file: `detect-debug-environment`, `sets-process-name`, `checks-cpu-name`, `android`, and `apk`. The tags `detect-debug-environment`, `checks-cpu-name`, and `sets-process-name` are explicit anti-analysis and VM evasion indicators. The `detect-debug-environment` tag indicates that the application actively probes for debugger and sandbox contexts at runtime, `checks-cpu-name` reveals hardware fingerprinting consistent with virtual machine detection, and `sets-process-name` indicates process name manipulation to obfuscate the running process from monitoring tools.

3.1.3 Relations

VirusTotal shows the contacted domains, URLs, and IP addresses associated with the APK. The five contacted URLs are all served over plain HTTP, which means their content can be intercepted or modified in transit:

- <http://busappmanager.elasticbeanstalk.com/v1/category/list.json>
- <http://busappmanager.elasticbeanstalk.com/v1/env/get.json>

Contacted URLs (5) ⓘ			
Scanned	Detections	Status	URL
2025-05-23	0 / 97	-	http://busappmanager.elasticbeanstalk.com/v1/category/list.json
2022-09-09	0 / 88	200	http://busappmanager.elasticbeanstalk.com/v1/env/get.json
2018-03-16	0 / 67	404	http://smartapps.cafe24.com/jjbus_android/environment.json
2018-03-16	0 / 67	200	http://busappmanager.elasticbeanstalk.com/v1/issue/my.json
2025-05-23	0 / 97	200	http://gwk.adlibr.com/ad/config/init

Contacted Domains (6) ⓘ			
Domain	Detections	Created	Registrar
adlibr.com	0 / 91	2012-04-04	Gabia, Inc.
busappmanager.elasticbeanstalk.com	0 / 91	-	-
cafe24.com	0 / 91	1999-05-21	Gabia, Inc.
elasticbeanstalk.com	0 / 91	2011-01-04	MarkMonitor Inc.
gwk.adlibr.com	0 / 91	2012-04-04	Gabia, Inc.
smartapps.cafe24.com	0 / 91	1999-05-21	Gabia, Inc.

Figure 3.3: VirusTotal Relations for Malbus_v3.6.5.apk

- `http://busappmanager.elasticbeanstalk.com/v1/issue/my.json`
- `http://smartapps.cafe24.com/jjbus_android/environment.json`
- `http://gwk.adlibr.com/ad/config/init`

The first three URLs are hosted on AWS Elastic Beanstalk (`busappmanager.elasticbeanstalk.com`) and appear to retrieve the app category list, runtime environment configuration, and remote issue data from a centralised backend. This mechanism is consistent with dropper-like behaviour, as it allows the app to modify its functionality without requiring a new APK release. The fourth URL (`smartapps.cafe24.com`) retrieves a configuration file specific to this `jjbus` variant, distinguishing it from the previous samples which contacted different regional `cafe24.com` endpoints. The domain `gwk.adlibr.com/ad/config/init` serves the advertising configuration and was already observed in the previous Malbus samples; it responds only to POST requests, otherwise returning `{"result":4003,"message":"key_media_key"}`.

The contacted domains are: `adlibr.com`, `busappmanager.elasticbeanstalk.com`, `cafe24.com`, `elasticbeanstalk.com`, `gwk.adlibr.com`, and `smartapps.cafe24.com`. No flagged domains were detected by VirusTotal at the time of analysis. No contacted IP addresses were reported directly in the Relations tab.

3.1.4 Behavior

VirusTotal's behaviour tab highlights several actions consistent with a suspicious application profile. The APK accesses file-related resources, builds JSON objects from filenames and absolute paths, reads data into buffer streams, and opens files from absolute paths. It also shows the use of reflection or control-based execution to start another application from the current one, which may indicate dynamic or indirect execution of external components. In addition, the app interacts with advertising and analytics libraries, scheduling recording tasks and reading files from the assets directory.

As detailed in the previous section, this variant carries explicit **anti-analysis and VM evasion** indicators (`detect-debug-environment`, `checks-cpu-name`, `sets-process-name`) not observed in `Malbus_v1.0.3.apk` but consistent with the pattern introduced in `Malbus_v3.3.7.apk`.

Overall, the observed behaviour suggests a complex application that performs file-system interaction, supports remote or dynamically driven actions, and is more evasion-aware than the previous Malbus variants. Additionally, a crowdsourced YARA rule match was recorded for this sample:

the IcedID structural rule from the CAPEv2 repository (author: kevoreilly/threathive) matched the binary, consistent with the pattern observed across all three Malbus variants.

3.2 Static Analysis

The Java source code was decompiled using JD-GUI / Bytecodeviewer and inspected manually to identify classes and methods of interest. Static analysis shows that the application combines legitimate bus-related functionality with advertising SDKs, remote configuration mechanisms, native code, and update-like behaviour. This indicates that the APK is not limited to its declared purpose, but may also support conditional execution and payload delivery at runtime.

3.2.1 MainActivity.java

- Inherits from `BaseMainActivity`.
- Calls `getTabView(BaseTabFragmentActivity.MainTabs.BUSTRANS).setVisibility(8)` to hide a UI tab.
- Provides ad-network keys through `getAdvertiseKey(...)`.
- Retrieves configuration through `getEnvironmentProperties(...)`.

This combination suggests that the visible interface is at least partially controlled at runtime. In particular, the hiding of a UI tab and the retrieval of external configuration parameters imply that the application may adapt its behaviour according to remotely supplied values. The use of multiple advertising providers, including AdMob, Firebase, and AdLib, further confirms the presence of monetization-oriented logic.

3.2.2 BaseMainActivity.java

- Acts as the central initialization hub.
- Loads the native library with `System.loadLibrary("Audio3.0")`.
- Declares and invokes the native methods `startUpdate()` and `updateApplication()`.
- Requests the runtime permissions:
 - `READ_EXTERNAL_STORAGE`
 - `WRITE_EXTERNAL_STORAGE`
 - `REQUEST_INSTALL_PACKAGES`
- Loads remote app environment data, including a URL, a database version, and a remote download URL.

This class is the most relevant part of the static analysis. The presence of a native library, together with methods related to update handling and the request for `REQUEST_INSTALL_PACKAGES`, is consistent with dropper-like behaviour. In addition, the remote environment loading mechanism allows the application to modify its behaviour without requiring a new release.

3.2.3 EnvConstant.java

The `EnvConstant` class defines several hardcoded advertising and analytics keys. MobSF confirmed the extraction of hardcoded secrets including `adpostkey`, `adlibkey`, encrypted strings, and hash keys embedded directly in the source code.

```

public static final String ADAMKEY = "3034Z1OT1385cf1db09";
public static final String ADLIB_KEY = "590ea88a0cf2843c93807ed9";
public static final String ADMOB_UNITID = "ca-app-pub-0388605861855854/7499990747";
public static final String FACEBOOK = "627046304133735685335228304842";
public static final String GA_TRACKER_ID = "UA-56668425-6";

```

These constants confirm the integration of multiple ad SDKs and tracking frameworks. Their presence in plaintext reduces transparency and can facilitate reverse engineering or unauthorised reuse of the embedded services.

3.2.4 BusApiWeb.java

BusApiWeb performs network requests to bus-related endpoints over plain HTTP. It uses Volley for networking, parses HTML responses instead of JSON, and relays information through listeners.

- Makes calls to <http://jeonjuits.go.kr> and related bus information endpoints.
- Uses Volley for networking.
- Parses HTML responses instead of JSON.
- Relays information through listeners.

The exclusive use of HTTP means that bus-stop identifiers and route data travel in plaintext and can be intercepted or modified in transit, creating a clear man-in-the-middle risk.

3.2.5 AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>
<manifest android:versionCode="74" android:versionName="3.6.5" package="lmh.android.jjbus" platformBuildVersionCode="24" platformBuildVersionName="7.0"
  xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-sdk android:minSdkVersion="14" android:targetSdkVersion="25" />
  <permission android:name="lmh.android.jjbus.permission.C2D_MESSAGE" android:protectionLevel="signature" />
  <uses-permission android:name="android.permission.INTERNET" />
  <uses-permission android:name="android.permission.ACCESS_LOCATION_EXTRA_COMMANDS" />
  <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
  <uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
  <uses-permission android:name="android.permission.CHANGE_WIFI_STATE" />
  <uses-permission android:name="android.permission.VIBRATE" />
  <uses-permission android:name="android.permission.WAKE_LOCK" />
  <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
  <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
  <uses-permission android:name="com.google.android.c2dm.permission.RECEIVE" />
  <uses-permission android:name="com.google.android.providers.gsf.permission.READ_GSERVICES" />
  <uses-permission android:name="com.android.browser.permission.WRITE_HISTORY_BOOKMARKS" />
  <uses-permission android:name="lmh.android.jjbus.permission.C2D_MESSAGE" />

```

Figure 3.4: AndroidManifest of Malbus_v3.6.5.apk

- Declares the following permissions:
 - INTERNET, ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION
 - READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE
 - REQUEST_INSTALL_PACKAGES
- Declares multiple ad and analytics activities:
 - AdLib, AdMob, Facebook, AdFit (Kakao), Crashlytics
- Includes GCM and Firebase messaging support.
- Declares the custom service `lmh.android.jjbus.GCMIntentService`.
- Exports 3 Activities, 3 Services, and 4 Receivers accessible to other applications.

The manifest confirms that the application can receive background messages and interact with external services. In particular, `REQUEST_INSTALL_PACKAGES` may allow silent installation of additional APKs, consistent with dropper-like behaviour.

3.2.6 AppInfoActivity.java

- Displays a public bus-related URL for the Jeonju city service.
- Serves to legitimize the app.
- Contains no interaction logic.

This activity appears to be mainly cosmetic. Its role seems to be presenting the application as a legitimate transportation service, even though the rest of the static analysis reveals a much more complex and suspicious architecture.

3.3 Summary

Suspicious Pattern	Explanation
Multiple ad SDKs	Monetization through aggressive advertising and possible ad fraud (AdMob, Firebase, AdLib, Crashlytics).
Native libraries (JNI)	<code>libAudio3.0.so</code> present for multiple architectures; lacks stack canary and fortify function protections.
<code>REQUEST_INSTALL_PACKAGES</code>	Suggests dropper-like behaviour and possible APK installation without market interaction.
Unencrypted HTTP endpoints	Data to <code>jeonjuits.go.kr</code> and AWS Elastic Beanstalk can be intercepted or modified through MITM attacks.
Dynamic configuration via <code>getEnvironmentProperties(...)</code>	The app can change behaviour based on remote values without a new release.
Fragment and tab visibility manipulation	Suggests conditional or delayed functionality and may hide behaviour from users and analysts.
Hardcoded secrets	API keys and encrypted strings embedded in source (<code>adpostkey</code> , <code>adlibkey</code> , hash keys).
Anti-analysis & VM evasion	Active evasion capabilities absent in <code>Malbus.v1.0.3.apk</code> but present in both <code>Malbus.v3.3.7.apk</code> and <code>Malbus.v3.6.5.apk</code> .

3.4 MobSF

MobSF assigned a security score of **38/100** (grade C, HIGH RISK) to this APK, confirming that the application exposes a broad attack surface. The scorecard shows a high concentration of High and Medium severity issues, alongside informational and secure findings. Problems span six categories: certificate, manifest, code, trackers, secrets, and permissions, with 8 privacy trackers detected out of 432 checked and 4 permission hotspots flagged as critical.

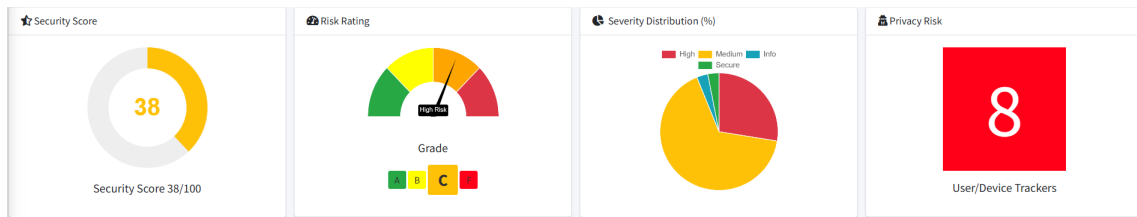


Figure 3.5: MobSF Scorecard Malbus_v3.6.5.apk

3.4.1 Certificate Analysis

During the MobSF analysis, one unique certificate was found. The binary is signed with the `v1` signature scheme only, while `v2`, `v3`, and `v4` are absent. The certificate uses `SHA1withRSA`, with subject and issuer both identified as `C=ko, L=Jeonju, CN=Myoung-Hun Lee`. The validity interval spans from 2011-05-25 to 2061-05-12, and the reported hash algorithm is SHA-1. This signing configuration is weak from a security perspective because it relies on legacy mechanisms associated with known cryptographic and platform-level weaknesses, making it vulnerable to the Janus attack on Android 5.0–8.0.

3.4.2 High Severity Findings

MobSF reports multiple high-severity issues across certificate, manifest, and code categories. From the certificate, the application is vulnerable to the Janus attack due to `v1`-only signing, and SHA-1 is used as the hash algorithm. From the manifest, the low minimum SDK version (`minSdk=14`) allows installation on older unpatched Android versions, and the StrandHogg 2.0 weakness is present in the Google app-invite preview activity.

From the code analysis, three further high-severity findings emerge: logging of sensitive information, world-readable files or `SharedPreferences`, and an active debug configuration. Additionally, the code uses an insecure random number generator and weak cryptographic primitives (MD5), both of which reduce the reliability of any security-sensitive operations performed by the application.

Finally, the shared library analysis reveals that `libAudio3.0.so`, compiled for multiple architectures, lacks both stack canary protection and fortify function hardening, making it susceptible to memory corruption attacks.

3.4.3 Medium Severity Findings

The medium-severity findings are mainly related to insecure configuration, component exposure, and storage. The flag `android:allowBackup=true` makes application data accessible through ADB backups when USB debugging is enabled.

Three Activity, three Service, and four Receiver components are exported without adequate protection, including components related to Firebase, GCM, and advertising frameworks. MobSF also warns about hardcoded secrets and API keys (`adpostkey`, `adlibkey`) found across multiple source files, and flags insecure WebView usage.

On the storage side, files and `SharedPreferences` are world-readable, and the application can read and write external storage, potentially exposing user data to other applications. The use of MD5 for hashing is also flagged as a medium-severity issue due to its known collision-resistance weaknesses.

3.4.4 Low Severity Findings

MobSF reports informational issues related to application logging, which may include sensitive data. It also reports the use of SSL communication and the presence of root detection capabilities as secure findings. The scorecard additionally flags four permission hotspots as critical, reinforcing the idea that the application is highly sensitive from a privacy perspective.

3.4.5 Overall Assessment

Overall, MobSF confirms that the APK is not a simple transportation application. The combination of a score of 38/100, weak signing, exported components, insecure native libraries without memory protections, hardcoded secrets, world-readable storage, and an extensive ad/tracking infrastructure indicates an application with high security risk. The capability to silently install additional packages further widens the attack surface beyond what is expected from a bus-information service.

3.5 Genymotion

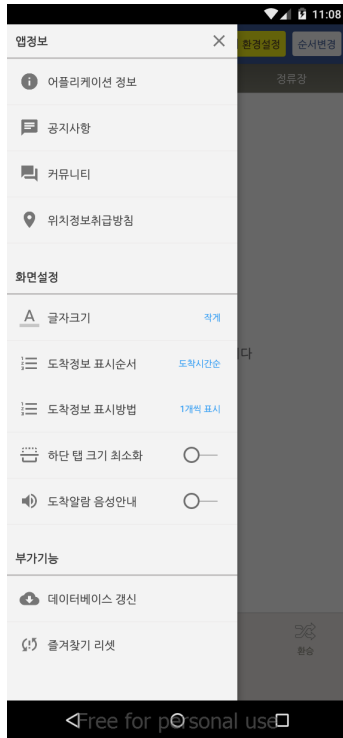
The dynamic analysis performed in Genymotion provided practical confirmation of several findings obtained during static inspection. The application interacts with remote endpoints, relies on advertising frameworks, and appears capable of adapting part of its behaviour through external configuration data rather than through local code changes. This behaviour is relevant because it indicates that the app is not limited to a simple transportation interface. Instead, it combines user-facing functionality with background services, ad-related activity, and network-driven logic, which is consistent with a repackaged or trojanized application structure. The emulator analysis therefore strengthens the overall assessment of the sample as suspicious. In combination with insecure HTTP traffic, exported components, hardcoded secrets, and multiple privacy trackers identified by other tools, the Genymotion observations support the conclusion that the APK presents a non-negligible security risk.

3.6 Conclusion

The combined results of VirusTotal, static analysis, MobSF, and Genymotion confirm that Malbus_v3.6.5.apk is malicious and constitutes the most capable variant of the Malbus family analysed in this report. The core dropper architecture, advertising SDK integration, and certificate weaknesses identified in the two earlier variants are fully preserved, but Malbus_v3.6.5.apk introduces a qualitatively distinct expansion of the attack surface through the Crashlytics SDK, the unprotected native library `libAudio3.0.so`, and an elevated number of exported components.

The MobSF score of **38/100** (Grade C, HIGH RISK) and the elevated count of high-severity findings reflect this expanded footprint: it is the only numerical risk rating explicitly produced across the three variants, underscoring the greater breadth of issues present in this sample. The shared library `libAudio3.0.so`, compiled without stack canary or fortify function protections, represents an additional attack surface absent from the previous variants and makes this sample more susceptible to memory corruption exploitation.

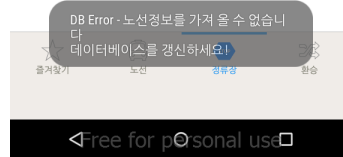
The evasion layer further distinguishes Malbus_v3.6.5.apk from Malbus_v1.0.3.apk: the VirusTotal tags `detect-debug-environment`, `checks-cpu-name`, and `sets-process-name` confirm that sandbox and debugger detection, first introduced in Malbus_v3.3.7.apk, are a stable feature of the later variants. Together, these characteristics indicate that Malbus_v3.6.5.apk was not merely maintained but actively developed to extend the malware's reach, making it the most forensically challenging sample in the Malbus family under analysis.



(a) Side menu showing app settings, display preferences, and the database refresh button (데이터베이스 갱신) under Additional Functions Malbus_v3.6.5.apk



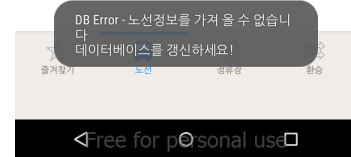
검색결과가 없습니다.



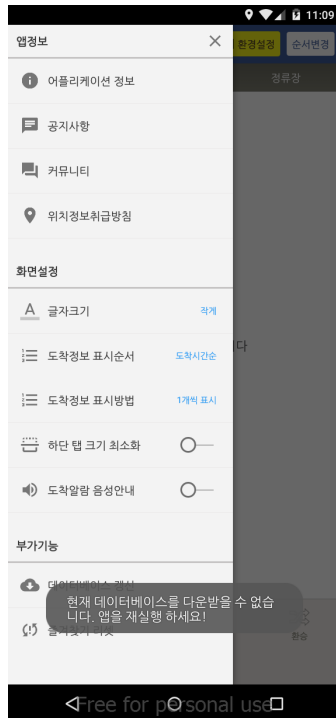
(b) Bus stop search screen showing a DB Error — the app cannot retrieve route information and prompts a database refresh Malbus_v3.6.5.apk



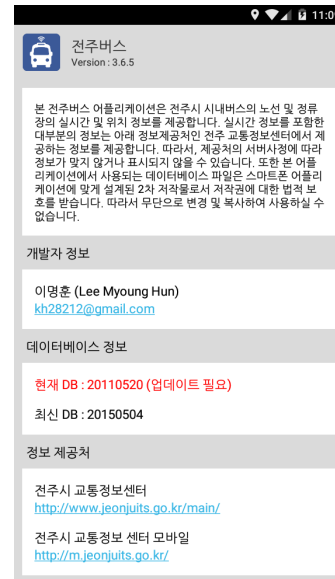
검색결과가 없습니다



(c) Bus line search screen showing the same DB Error, confirming that neither route nor stop data can be retrieved without a database update Malbus_v3.6.5.apk



(d) Side menu with a new error popup: the database download failed and the app requests a restart Malbus_v3.6.5.apk



(e) App info screen for Jeonju Bus v3.6.5 highlighting the outdated local database (20110520, highlighted in red) against the latest available version (20150504) Malbus_v3.6.5.apk

Figure 3.6: Gynemtion dynamic analysis screenshots of Malbus_v3.6.5.apk

Chapter 4

Comparative Analysis

This chapter provides a cross-sample comparison of the three Malbus APK variants analysed in the previous chapters: **Malbus Malbus_v1.0.3.apk** , **Malbus Malbus_v3.3.7.apk** , and **Malbus Malbus_v3.6.5.apk** . The goal is to identify common patterns, progressive evolution of malicious capabilities, and differences in evasion sophistication across the family.

4.1 VirusTotal Detection

All three samples were submitted to VirusTotal and flagged as malicious by a comparable number of security engines. The suggested threat label assigned by VirusTotal is identical across all samples — **trojan.andr/apkdownloader** — and the Tencent engine correctly identified each sample by referencing its specific APK package name (**a.privacy.MalBuslmh**).

Metric	Malbus_v1.0.3.apk	Malbus_v3.3.7.apk	Malbus_v3.6.5.apk
Malicious detections	40/75	40/75	38/75
Reputation score	−1	−2	0
Suggested threat label	trojan.andr/apkdownloader		
Dominant engine category	trojan (18)	trojan (17)	trojan (17)
Secondary engine category	downloader (5)	downloader (4) + dropper (1)	downloader (5)
Total submissions	377	354	—
Alternative filenames	Yes	Yes	Yes

Table 4.1: VirusTotal detection summary across the three APK samples

Notably, well-known vendors such as Microsoft Defender, McAfee, and Malwarebytes failed to detect any of the three samples as malicious, highlighting a significant gap in coverage for this malware family.

4.2 Permissions and Attack Surface

All three APKs share a core permission set consistent with the same malware family. The most security-relevant permissions present across all samples are:

- **ACCESS_FINE_LOCATION** and **ACCESS_COARSE_LOCATION** — device tracking via GPS and cell towers.
- **READ_EXTERNAL_STORAGE** / **WRITE_EXTERNAL_STORAGE** — access to user files on external memory.
- **REQUEST_INSTALL_PACKAGES** — silent installation of additional APKs, a key dropper indicator.

- **INTERNET** — unrestricted network access, used for remote configuration and payload delivery.
- **GCM/FCM** push messaging (**C2DMESSAGE**, **C2DM.RECEIVE**) — allows remote triggering of behaviour.

Two per-version differences emerge when comparing the manifests. First, **WRITE_HISTORY_BOOKMARKS** and **READ_GSERVICES** are present in **Malbus_v1.0.3.apk** and **Malbus_v3.6.5.apk**, but are absent in **Malbus_v3.3.7.apk**. The **WRITE_HISTORY_BOOKMARKS** permission grants read and write access to the Android stock browser’s history and bookmarks. Its presence in two out of three variants is not consistent with a legitimate bus-information application and suggests either browser history exfiltration for user profiling — a capability observed in banking-style trojans that fingerprint visited financial sites — or click-fraud redirection, in which the malware inserts attacker-controlled bookmarks to inflate advertising revenues. The permission’s absence in **Malbus_v3.3.7.apk** may reflect an intentional reduction of the detectable attack surface for that release, subsequently reversed in **Malbus_v3.6.5.apk**. Second, **Malbus_v3.6.5.apk** uniquely declares **CALL_PHONE**, **RECORD_AUDIO**, and **READ_PHONE_STATE** in its manifest, permissions entirely absent from **Malbus_v1.0.3.apk** and **Malbus_v3.3.7.apk**. Combined with the Crashlytics SDK declared only in **Malbus_v3.6.5.apk**, this indicates a measurably expanded attack surface in the latest variant.

4.3 Behavioural Tags and Evasion Capabilities

A clear evolution in anti-analysis sophistication is observable across the three versions. While **Malbus_v1.0.3.apk** employs obfuscation and Java Reflection, versions **Malbus_v3.3.7.apk** and **Malbus_v3.6.5.apk** introduce explicit VM and debugger evasion mechanisms. Noteworthy is that **Malbus_v3.6.5.apk** only carries the three anti-analysis tags and lacks the **obfuscated**, **reflection**, **telephony**, and **checks-gps** tags visible in **Malbus_v1.0.3.apk** and **Malbus_v3.3.7.apk**; this likely reflects a difference in VirusTotal sandbox coverage for this sample rather than the removal of those capabilities from the code.

Behavioural Tag	Malbus_v1.0.3.apk	Malbus_v3.3.7.apk	Malbus_v3.6.5.apk
obfuscated	✓	✓	×
reflection	✓	✓	×
checks-gps	✓	✓	×
telephony	✓	✓	×
detect-debug-environment	×	✓	✓
checks-cpu-name	×	✓	✓
sets-process-name	×	✓	✓

Table 4.2: VirusTotal behavioural tags across the three APK samples

This progression confirms that the malware author actively improved the evasion layer between **Malbus_v1.0.3.apk** and the later variants, adding sandbox detection and process name manipulation to harden the payload against dynamic analysis environments.

4.4 Static Analysis: Common Architecture

The static analysis of all three samples reveals a shared internal architecture, confirming they are regional variants of the same malware family. The core classes — **MainActivity**, **BaseMainActivity**, **EnvConstant**, **BusApiWeb**, **AppInfoActivity** — are present and functionally equivalent across all three APKs, differing only in regional bus endpoint URLs and package names (**cwbus**, **gjbus**, **jjbus**).

The following structural elements are common to all three samples:

- Loading of a native library via **System.loadLibrary("Audio3.0")** in **BaseMainActivity**.

- Native methods `startUpdate()` and `updateApplication()`, consistent with dropper-like update logic.
- Remote environment loading mechanism (`getEnvironmentProperties(...)`) that allows runtime behaviour modification without a new APK release.
- Hardcoded advertising and analytics keys in `EnvConstant`: AdMob, AdLib, Facebook, Google Analytics (UA-56668425-6).
- Unencrypted HTTP communication in `BusApiWeb`, exposing bus-stop data to man-in-the-middle interception.
- A cosmetic `AppInfoActivity` displaying a legitimate regional bus URL to disguise the application’s true nature.

An SDK-level difference is worth noting: Malbus_v3.6.5.apk additionally includes the **Crashlytics** crash-reporting SDK and references **Firebase** more prominently in `MainActivity`, while Malbus_v1.0.3.apk and Malbus_v3.3.7.apk rely on AdFit (Kakao) without Crashlytics.

4.5 C2 Infrastructure Status

As part of the dynamic analysis, the primary C2 endpoint shared by all three variants, `busappmanager.elasticbeanstalk.com`, was directly probed via `curl` requests to the known API paths (`/v1/env/get.json`, `/v1/category/list.json`, `/v1/issue/my.json`). The server returned no response on any path: the host did not resolve to an active IP address, confirming that the AWS Elastic Beanstalk environment hosting the Malbus distribution infrastructure has been de-commissioned.

This finding has two important implications. First, the dropper mechanism documented in `BaseMainActivity` — remote environment loading via `getEnvironmentProperties(...)`, followed by payload download and silent installation — could not be triggered end-to-end during dynamic analysis, because the C2 backend is no longer operational. The infection chain stages 2 through 4 described in the McAfee Malbus report (payload request, delivery, and installation of the second-stage trojan) are therefore confirmed at the structural and static level only; live execution was not possible. Second, the C2 inactivity explains the database download errors observed in Genymotion across all three samples: since the remote environment configuration cannot be retrieved, the application is unable to fetch its bus database or payload URL, and fails with a network error on startup.

The `smartapps.cafe24.com` variant-specific endpoints (`/gjbus_android/environment2.json`, `/cwbus_android/`, `/jjbus_android/environment.json`) were independently verified as unreachable at the time of analysis (June 2026), confirming that the entire Malbus remote configuration infrastructure is defunct.

4.6 MobSF Static Analysis Comparison

MobSF confirms consistent and severe security issues in all three samples. All APKs share the same certificate signing profile: SHA-1 with RSA, v1 scheme only, issued by `CN=Myoung-Hun Lee` — making all three vulnerable to the Janus attack on Android 5.0–8.0. Notably, Malbus_v3.6.5.apk received an explicit MobSF risk score of **38/100 (Grade C, HIGH RISK)**, encompassing 8 privacy trackers detected out of 432 checked and 4 critical permission hotspots; the other two samples were not assigned a numerical score in the analysed report.

4.6.1 High-Severity Findings

Finding	Malbus_v1.0.3.apk	Malbus_v3.3.7.apk	Malbus_v3.6.5.apk
Janus vulnerability	✓	✓	✓
SHA-1 weak hashing (certificate)	✓	✓	✓
Low minSdk = 14	✓	✓	✓
Task Hijacking & StrandHogg	✓	✓	StrandHogg 2.0
Debug configuration enabled	✓	✓	✓
Weak encryption in AdLib library	✓	✓	×
SharedPreferences	✓	✓	✓
Privacy trackers (>8)	✓	✓	✓
Logging sensitive info ¹	×	×	✓
Insecure random number generator	×	×	✓
Weak cryptographic primitive	×	×	✓
libAudio3.0: no stack canary	×	×	✓
<i>Total HIGH findings</i>	<i>9</i>	<i>9</i>	<i>10</i>

Table 4.3: MobSF high-severity findings across the three APK samples

The total count of high-severity findings is 9 for Malbus_v1.0.3.apk and Malbus_v3.3.7.apk , and rises to **10** for Malbus_v3.6.5.apk . The composition shifts as follows: the weak AdLib encryption finding (absent in Malbus_v3.6.5.apk) is replaced by four code-level issues — logging of sensitive information (elevated from LOW in earlier variants), insecure RNG, weak MD5 cryptography, and the unprotected native library — while the full Task Hijacking weakness is reduced to StrandHogg 2.0 only.

4.6.2 Medium-Severity Findings

Finding	Malbus_v1.0.3.apk	Malbus_v3.3.7.apk	Malbus_v3.6.5.apk
android:allowBackup=true	✓	✓	✓
Exported / unprotected components	✓	✓	✓
INSTALL_PACKAGES permission	✓	✓	✓
Read/write external storage	✓	✓	✓
Hardcoded secrets (API keys)	✓	✓	✓
Insecure WebView usage	✓	✓	✓
Insecure RNG ²	✓	✓	✓
MD5 weak hashing ³	✓	✓	✓
SQL injection via raw queries	✓	✓	✓
CALL_PHONE / RECORD_AUDIO permissions	×	×	✓

Table 4.4: MobSF medium-severity findings across the three APK samples

For Malbus_v3.6.5.apk , MobSF reports Insecure RNG and MD5 at *both* medium and high severity, reflecting their dual presence across the scorecard sections of Chapter 3. The unique presence of CALL_PHONE, RECORD_AUDIO, and READ_PHONE_STATE in Malbus_v3.6.5.apk was also flagged as

¹Classified as a LOW-severity (informational) finding in Chapters 1 and 2; elevated to HIGH severity by MobSF in Chapter 3 for Malbus_v3.6.5.apk .

²Also flagged as HIGH severity for Malbus_v3.6.5.apk (see Table 4.3); MobSF reports it at both severity levels in Chapter 3.

³Also flagged as HIGH severity for Malbus_v3.6.5.apk (see Table 4.3); MobSF reports it at both severity levels in Chapter 3.

a medium-severity concern, consistent with the expanded manifest permissions discussed in Section 4.2.

4.7 YARA Rule Matches

All three samples matched the IcedID structural YARA rule from the CAPEv2 repository (author: [kevoreilly/threathive](#)). Although IcedID is historically associated with a Windows banking trojan, the matched rule targets dropper and loader architectures, not the specific malware family. This consistent structural match across all three variants reinforces the hypothesis that the Malbus family was deliberately engineered with an advanced payload-delivery architecture designed to withstand analysis and deliver second-stage payloads on demand.

4.8 Overall Family Assessment

The comparative analysis confirms that Malbus Malbus_v1.0.3.apk , Malbus_v3.3.7.apk , and Malbus_v3.6.5.apk are three regional variants of the same trojanised application family, sharing an identical malicious core while adapting the legitimate-facing functionality to different South Korean cities (Changwon, Gwangju, Jeonju).

The evolution across versions is twofold. First, anti-analysis and VM evasion capabilities are absent in Malbus_v1.0.3.apk but fully present in Malbus_v3.3.7.apk and Malbus_v3.6.5.apk (`detect-debug-environment`, `checks-cpu-name`, `sets-process-name`). Second, Malbus_v3.6.5.apk introduces additional hardware-level permissions (`CALL_PHONE`, `RECORD_AUDIO`, `READ_PHONE_STATE`), the Crashlytics SDK, and a native library (`libAudio3.0.so`) compiled without stack canary or fortify function protections — all absent in the earlier samples.

In all other dimensions — certificate signing, dropper architecture, remote configuration, advertising SDK integration, and the core MobSF finding set — the three samples are functionally identical, indicating a mature and deliberately maintained malware family rather than opportunistic or independent variants. The confirmed inactivity of the shared C2 infrastructure (see Section 4.5) prevented end-to-end reproduction of the second-stage payload delivery; the dropper mechanism is nonetheless fully evidenced at the structural and static analysis level across all three variants.

List of Figures

1	Malbus infection chain as documented by McAfee Mobile Research	4
1.1	VirusTotal summary analysis of file Malbus_v1.0.3.apk	6
1.2	VirusTotal Detection tab for Malbus_v1.0.3.apk	7
1.3	Android Permission of Malbus_v1.0.3.apk	8
1.4	Android Manifest Malbus_v1.0.3.apk	9
1.5	VirusTotal Relations Tab Malbus_v1.0.3.apk	9
1.6	MobSF Scorecard Malbus_v1.0.3.apk	13
1.7	Genymotion dynamic analysis screenshots of Malbus_v1.0.3.apk	15
2.1	VirusTotal Summary for Malbus_v3.3.7.apk	17
2.3	VirusTotal Relations for Malbus_v3.3.7.apk	19
2.4	AndroidManifest of Malbus_v3.3.7.apk	22
2.5	MobSF Scorecard Malbus_v3.3.7.apk	23
2.6	Genymotion dynamic analysis screenshots of Malbus_v3.3.7.apk	25
3.1	VirusTotal Summary for Malbus_v3.6.5.apk	27
3.2	VirusTotal Detection for Malbus_v3.6.5.apk	29
3.3	VirusTotal Relations for Malbus_v3.6.5.apk	30
3.4	AndroidManifest of Malbus_v3.6.5.apk	32
3.5	MobSF Scorecard Malbus_v3.6.5.apk	34
3.6	Genymotion dynamic analysis screenshots of Malbus_v3.6.5.apk	36